CE 4011 Static and Modal Structural Analysis Desktop MVP

Final Project Report

Mohammad Umair Naeem
2416055

Spring Semester 2025–2026

Abstract

This report documents a Python educational structural analysis desktop application for CE 4011. The final submitted desktop MVP supports two-dimensional Static Analysis and Modal Analysis through a Tkinter model-building workflow, XML save/load/export, educational intermediate result tables, matplotlib visualizations, and automated validation tests. Response Spectrum Analysis and Time-History Analysis backend work is retained only as deferred future-extension work and is not presented as part of the submitted desktop MVP.

Project repository can be accessed using the link below: umairnaeem2703/CE4011-Final-Project-Submission.

The project presentation can be accessed here: https://youtu.be/kjtyVTrM1Io.

Contents

1 Introduction

The CE 4011 term project requires structural analysis software with a computational engine, a usable input workflow, post-processing tools, object-oriented design, and validation. This project implements that requirement as a Python Tkinter desktop MVP for educational two-dimensional Static Analysis and Modal Analysis.

The software is designed for transparent learning rather than commercial-scale production modeling. It exposes the model data, DOF map, stiffness and mass matrices, reduced systems, force vectors, displacement and reaction results, member force recovery, N/V/M diagram data, and modal participation quantities so students can trace how idealized structures become numerical results.

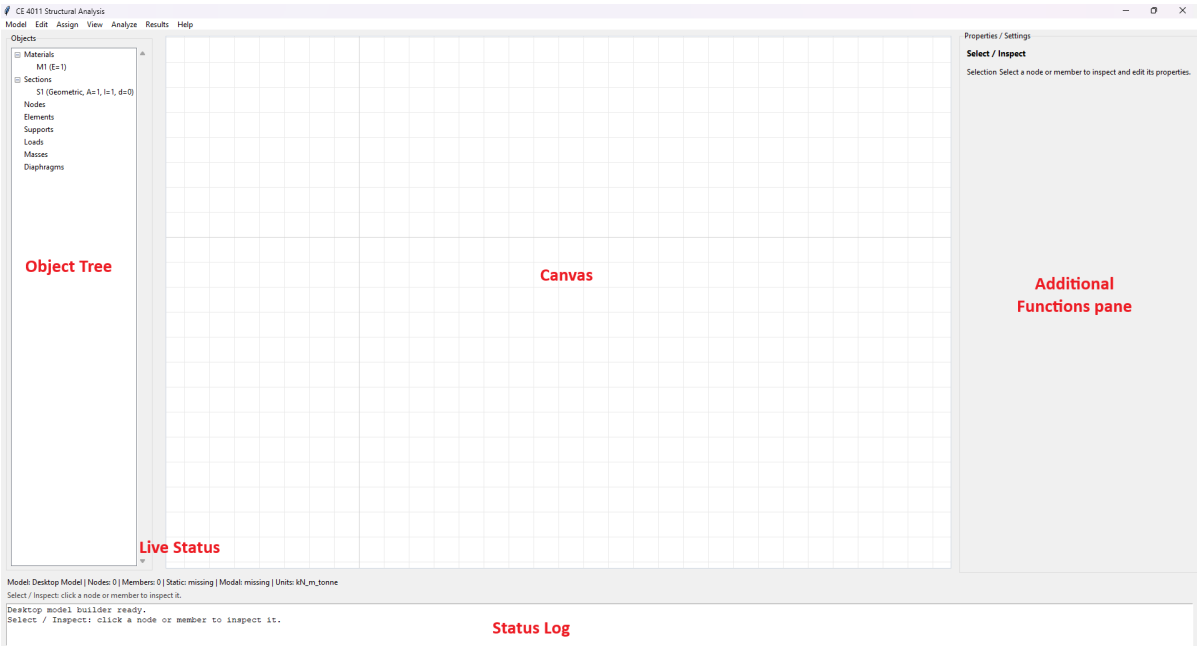

Figure 1: Main UI

The final submitted desktop scope is Static Analysis and Modal Analysis only. Response Spectrum Analysis and Time-History Analysis may remain in the repository as deferred future work, but they are not claimed as submitted desktop workflows. Detailed derivations, screenshot evidence, and benchmark tables are moved to the verification appendix to keep this main report within the CE 4011 main-body limit.

2 Scope of the Software

The submitted software supports two-dimensional frame, truss, beam-style, shear-frame, cantilever, and benchmark models through one common model representation. These are not separate solver families; they are different configurations of nodes, elements, releases, supports, properties, loads, and masses that pass through the same assembly and solver pipeline.

The final desktop scope includes Tkinter model creation and editing, XML load/save/export reproducibility, Static Analysis, Modal Analysis, educational result tables, matplotlib plots, and visible result export where implemented. Static results include displacements, reactions, member-end forces, and axial/shear/moment recovery. Modal results include eigenvalues, frequencies, periods, mode shapes, modal masses, participation factors, effective modal masses, and mass participation ratios.

The project does not claim three-dimensional analysis, nonlinear analysis, design-code checks, staged construction, automated meshing, or production-grade commercial modeling. RSA and THA are future-extension topics only. The accompanying installation manual explains how to run the application, and the user manual documents the end-user workflow and example usage.

3 Theoretical Background

Each two-dimensional node uses the DOF vector $[u_x, u_y, r_z]$. A two-node frame element therefore carries $[u_{x,i}, u_{y,i}, r_{z,i}, u_{x,j}, u_{y,j}, r_{z,j}]$. Truss behavior uses the translational components needed for axial deformation

while sharing the same global model DOF map. Element local quantities are transformed to global coordinates using $q_l = Tq_g$, $k_g = T^T k_l T$, and, for modal analysis, $m_g = T^T m_l T$.

Static analysis follows the direct stiffness method. Element stiffness and equivalent nodal loads are assembled into the global system $Ku = F$. Boundary conditions partition the system into free and restrained DOFs:

$$K_{ff}u_f = F_f - K_{fr}u_r.$$

After solving for u_f, the full displacement vector is reconstructed and reactions are recovered from $R = Ku - F$. Member-end forces are recovered in local coordinates using element displacements and fixed-end effects. The post-processor uses the recovered member forces and load data to generate axial force N, shear force V, and bending moment M diagram values.

Modal analysis solves the generalized eigenvalue problem

$$K\phi = \lambda M\phi,$$

where $\lambda = \omega^2$, $f = \omega/(2\pi)$, and $T_p = 1/f$. Mass is assembled from modeled member and nodal mass data, with lumped mass used as the final educational default. Disconnected zero-mass DOFs may be removed, but stiffness-coupled massless DOFs are statically condensed:

$$K_{\text{eff}} = K_{aa} - K_{am}K_{mm}^{-1}K_{ma}, \qquad M_{\text{eff}} = M_{aa}.$$

Mode shapes are mass-normalized. For influence vector r, the participation factor is $\Gamma_n = (\phi_n^T Mr)/(\phi_n^T M\phi_n)$, the effective modal mass is $M_{\text{eff},n} = \Gamma_n^2(\phi_n^T M\phi_n)$, and the participation ratio is $M_{\text{eff},n}/(r^T Mr)$. Detailed stiffness matrices, derivations, and benchmark equations are retained in the verification appendix.

4 Software Architecture and Object-Oriented Design

The architecture follows one object-oriented pipeline:

Tkinter canvas/tables/templates/XML
→ `ModelBuilder` → `StructuralModel` → DOF/K/M/F assembly
→ Static or Modal solver → result object → tables/plots/export

This design keeps responsibilities separated. `ModelBuilder` creates models from programmatic calls, templates, XML, tables, and desktop actions. `StructuralModel` stores nodes, elements, materials, sections, supports, loads, masses, releases, and validation state; it does not solve or plot. `StructuralValidator` centralizes validity checks. DOF and assembly modules create K, M, F, and reduced systems. Static and Modal solvers operate on assembled matrices and return result objects. UI and controller code coordinate user actions and display results without owning solver math.

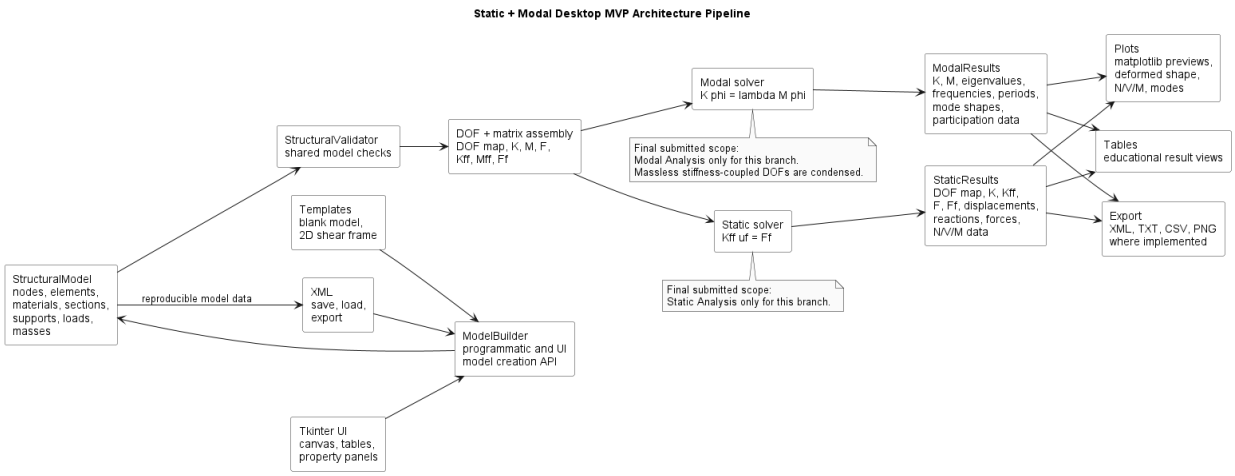

Figure 2: Architecture pipeline

Frames, trusses, shear frames, cantilevers, and benchmark problems all become ordinary model instances before assembly. This avoids duplicate solver logic by model family and makes validation stronger because each model type exercises the same Static and Modal computational path.

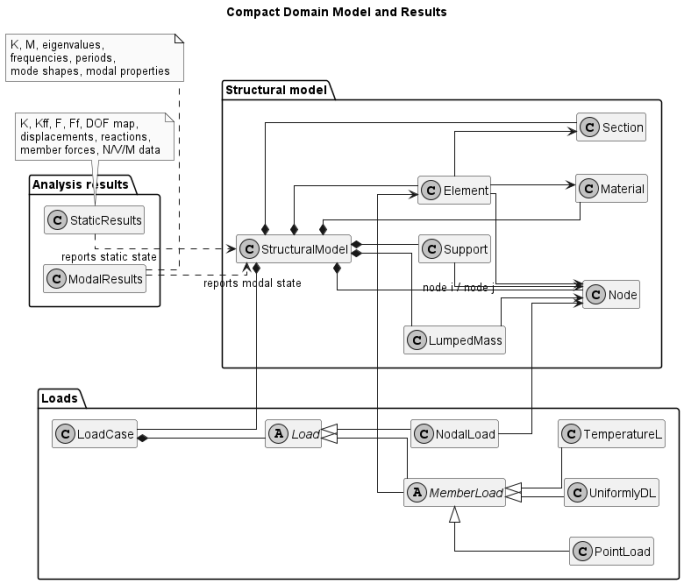

Figure 3: Domain model classes

5 Input and Output Structure

The user-facing input path is the Tkinter desktop workflow. A student can start from a blank model or shear-frame template, define nodes and members on the canvas, assign material and section properties, apply supports and loads, assign masses for modal analysis, validate the model, and then run Static or Modal analysis. The object tree and property panel support editing without writing XML or source code.

XML is the reproducibility backend. A desktop-created model can be saved/exported to XML and later loaded into the same `ModelBuilder` and `StructuralModel` path. The XML stores analysis-relevant geometry, materials, sections, supports, releases, loads, masses, unit settings, and load cases, while ordinary users work through the desktop UI.

Static outputs include the DOF map, K, K_{ff}, F, F_f, displacements, reactions, member-end forces, and N/V/M data. Modal outputs include K, M, reduced or condensed matrices, eigenvalues, frequencies, periods, mode shapes, modal masses, participation factors, effective modal masses, and mass participation ratios. Visible tables and plots can be exported where implemented. The installation manual covers environment setup, and the user manual gives a complete workflow example.

6 Analysis Procedure

6.1 Static Analysis Workflow

Static analysis begins after the model validates successfully. The desktop command obtains the current model from `ModelBuilder`; `StructuralValidator` checks the model; the DOF optimizer identifies free and restrained DOFs; the assembler forms K, F, K_{ff}, and F_f; the static solver computes free displacements; and the post-processor reconstructs full displacements, reactions, member forces, and N/V/M data. The final `StaticResults` object feeds the tables, plots, and export functions.

This sequence preserves the educational path from user input to reduced system and recovered engineering quantities. It also keeps solver calculations outside the UI layer.

6.2 Modal Analysis Workflow

Modal analysis uses the same validated model but adds mass assembly and dynamic DOF handling. The stiffness and mass matrices are assembled, active dynamic DOFs are selected, disconnected zero-mass DOFs may be removed, and stiffness-coupled massless DOFs are condensed before the eigenproblem is solved. `ModalSolver` returns eigenvalues, frequencies, periods, normalized mode shapes, participation factors, effective modal masses, and participation ratios in a `ModalResults` object for display and export.

7 Result Visualization and Post-Processing

Post-processing is provided through Tkinter result windows and matplotlib plots. Plotting functions consume the model and result objects; they do not perform solver math. This separation keeps each plotted result traceable to `StaticResults`, `ModalResults`, and post-processor data.

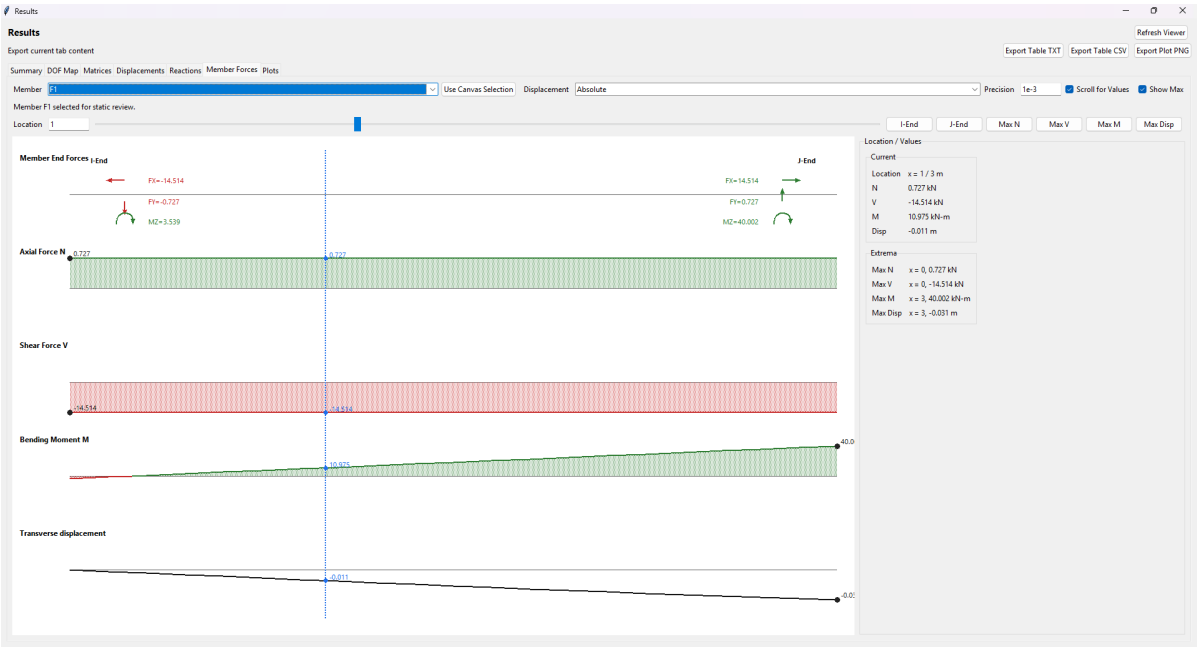

Figure 4: Results and visualization

The main visual outputs are model preview, static deformed shape, axial/shear/moment diagrams (with individual member intermediate internal forces and displacement inspection), and modal mode shapes. Result tables provide the numerical counterpart: matrices, vectors, DOF labels, displacements, reactions, member forces, diagram values, eigenvalues, frequencies, periods, mode-shape components, and participation quantities. Visible result tables can be exported as text or CSV where implemented, visible plots can be exported as PNG where implemented, and XML remains the reproducible model format.

8 Testing and Validation Summary

The final compact testing baseline for the submitted scope is `pytest tests/ -q` → 197 passed. Detailed focused test commands, terminal evidence, benchmark tables, and reference comparisons are placed in the verification appendix.

Table 1: Compact validation summary

Area	Evidence kept in the final baseline	Status
Static analysis	DOF mapping, stiffness assembly, boundary reduction, displacements, reactions, member forces, N/V/M data, and SAP2000/reference benchmarks.	Passed
Modal analysis	Mass assembly, massless DOF handling, eigenvalues, frequencies, periods, mode shapes, participation factors, effective mass, and participation ratios.	Passed
Desktop workflow	Tkinter Static + Modal command paths, XML save/export parse-back, and TXT/CSV/PNG result export paths.	Passed
Visualization and XML	Model/result plotting adapters, result table formatting, XML round trips, and reproducibility checks.	Passed

Final hardening changed only `tests/test_ui_desktop_static.py`; no solver math, XML schema, fixtures, or source behavior changed. Static benchmark tests use retained `data/test-frame.xml`. `data/ground_motion.txt` was restored only for legacy/future THA backend compatibility. The final P0 desktop tests covered XML save/export command paths and TXT/CSV/PNG result export paths.

9 Limitations and Possible Improvements

The submitted desktop MVP is intentionally limited to two-dimensional Static Analysis and Modal Analysis. It is an educational tool rather than a commercial structural analysis package. It does not claim three-dimensional modeling, nonlinear analysis, design-code checking, staged construction, automated meshing, advanced load combinations, production-quality drawing tools, or final desktop RSA/THA workflows.

Main limitations are the two-dimensional DOF set $[u_x, u_y, r_z]$, the educational lumped-mass-oriented modal workflow, the deliberately transparent standard-library numerical core, and remaining placeholder figures/tables that must be replaced before final packaging. More advanced modelling features were either not implemented or only partly explored because they were not necessary for the submitted Static and Modal scope. These include translational and rotational spring supports, member axis transformation options, inclined supports, rigid end zones, three-dimensional model generation, torsional deformation, and full three-dimensional member transformation.

Possible improvements include a polished installer, richer classroom examples, automated report generation, additional benchmark cases, expanded plot styling, more robust input checking, and broader modal modeling options. The omitted or partial modelling features could also be implemented in future work to investigate different structural cases, especially models with support flexibility, skewed boundary conditions, member end offsets, torsion-sensitive behaviour, or three-dimensional framing action. RSA and THA should be promoted to desktop workflows only if the project scope is formally expanded, and any future feature should preserve the same model-to-assembly-to-solver-to-result-object-to-visualization architecture.

10 Conclusion

The project delivers a CE 4011 desktop MVP for transparent two-dimensional Static and Modal structural analysis. The Tkinter workflow supports model creation, property assignment, validation, XML save/load/export, analysis execution, educational result tables, matplotlib visualization, and result export where implemented.

The main technical contribution is the consistent object-oriented pipeline: `ModelBuilder` creates the model, `StructuralModel` stores model data, validators and assemblers prepare numerical systems, solvers operate on assembled matrices, and result objects feed post-processing, plots, tables, and exports. This keeps solver math out of UI code and allows all submitted model types to exercise the same Static and Modal path.

The final reported baseline of 197 passing tests supports the submitted numerical and workflow claims. The accompanying verification appendix provides the detailed screenshot evidence, benchmark tables, and exported result evidence supporting the submitted MVP.

References

[1] CE 4011 Course Staff. Ce 4011 term project requirements, 2026. Special Topics in Civil Engineering: Structural Analysis Software Development, Spring Semester 2025–2026.

[2] Project Repository. Architecture documentation for ce 4011 static + modal mvp, 2026. Repository root ARCHITECTURE.md.

[3] Project Repository. Ce 4011 static + modal structural analysis mvp readme, 2026. Repository root README.md.

[4] Project Repository. Mathematical specification for static and modal desktop mvp, 2026. Repository root MATH_SPEC.md.